第 1 页：从梯度下降到共轭梯度

今天我们从一个具体问题出发：一个区域被划分成很多网格以后，怎样高效求出每个位置的未知量？这个问题会带我们走到一个稀疏线性系统，再走到梯度下降、共轭梯度和预条件。

我希望大家最后能把 3 件事联系起来。第 1，矩阵的特征值怎样影响迭代速度；第 2，共轭梯度怎样利用已经完成的搜索；第 3，怎样把问题本身的结构变成更快的求解方法。最后我们还会看实际代码、计算成本和结果检查，让这条路线落到可以运行的求解过程上。

第 2 页：从一张稳态场，得到一个线性系统

先看一个定义在单位正方形上的稳态问题。未知函数是 u，右边的 f 是源项，四周边界上的 u 都等于 0。方程里，横向和纵向的二阶导数系数不同。这里 epsilon 取 0.01，所以纵向的耦合比横向弱。

左图画的是源项，右图画的是网格上的数值解。为了能直接检查离散过程，我们先选择一个已知的正弦函数作为解析解，再把它代回微分方程，计算对应的源项。这样，离散求出来的场就有一个明确的比较对象。

每个方向取32 个内部点，总共是1024 个未知量。右图数值解与解析解的最大差大约是7.54×10^{-4}。这里比较的是网格离散结果与连续问题的解析解。

现在我们已经看到了要求的场。下一步是把这个连续方程变成有限个未知数之间的关系。

第 3 页：一个五点模板，连接上千个未知量

看左上的五点模板。中心点与左右2 个邻点通过系数-1 连接，与上下2 个邻点通过负 epsilon 连接。中心的系数是$2(1+\epsilon)$。这些系数来自对二阶导数的中心差分近似。

把所有网格值按行排成向量，就得到 L x 等于 b。右端 b 包含网格步长平方与源项的乘积。式子里的 T 是1 维三对角矩阵，Kronecker 积把1 维耦合复制到2 维网格的各行、各列。

右边的矩阵图说明了这种局部连接的结果。虽然矩阵有 1000 多行、1000 多列，每行主要只和少数邻点发生关系。按当前编号排列，非零元素集中在 5 条带上；局部放大图还能分辨不同方向的系数。

这种结构直接影响计算量。每个方向64 个内部点时，未知量是 4096，非零元素只有20224 个。稠密存储需要128 MiB，而这里的 CSR 存储大约只要0.247 MiB。后面的算法会反复计算矩阵乘向量，稀疏结构让这一步的成本与非零元素数量成正比。

第 4 页：解方程等价于最小化二次能量

接下来把线性方程换成一个优化问题。我们考虑对称正定矩阵：对称表示 A 等于它的转置，正定表示任意非零方向上的二次型都严格为正。

对这样的矩阵，可以定义页面上的二次函数。它的梯度是 A x 减 b，因此梯度为 0 的点恰好满足 A x 等于 b。这就把解线性方程和寻找二次函数的最低点联系起来。

我们要区分2 个量。残差 r 等于 b 减 A x，能直接从当前解和原方程计算；误差 e 等于真解减当前解，表示我们离真解有多远。对于这个二次函数，函数值高出最低点的部分，恰好等于误差的 A 范数平方的 1/2。

右图把这种关系画成等高线。星号是最低点，椭圆的方向由特征向量决定，沿不同方向的陡峭程度由特征值决定。页面下方给出一个2 维系统，真解是 1/11 和 7/11。后面几页都用这个小例子，把抽象公式一步步算出来。

第 5 页：沿负梯度走，步长由1 维最小化确定

有了能量的解释，最直接的策略就是沿负梯度走。因为残差正好是负梯度，所以更新方向可以取 r。剩下的问题是：沿这个方向走多远?

把 x 加上 alpha 乘 r 代入目标函数，就得到只关于 alpha 的1 维函数。对 alpha 求导，第 1 项是负的 r 内积 r，第 2 项是 alpha 乘 r 内积 A r。令导数等于 0，就得到页面中间的步长公式。

看右边的曲线。对于刚才的2 维系统，初始点取 0，初始残差就是 1 和 2 组成的向量。分子是 5，分母是 20，所以最优步长是 1/4。曲线上的最低点与这个计算完全对应，第1 步的新位置就是 1/4、1/2。

这一操作还有一个重要结果：走完之后，新残差与刚才的搜索方向正交。也就是说，我们已经把当前这条直线上的能量降到了最低。接下来要问的是，为什么每一步都选到直线上的最优位置，整体收敛仍然可能很慢?

第 6 页：条件数控制最速下降的收敛尺度

关键在不同方向的曲率差异。这里用最大特征值除以最小特征值定义条件数 kappa。条件数越大，能量等高线就可能越狭长。

左图展示条件数为 20 时的前15 步。迭代在狭长谷底两侧来回转向，每一步都完成了当前方向的精确线搜索，沿谷底前进的幅度却比较有限。

右图把这种现象量化。纵轴是相对于初始值的能量误差，采用对数尺度。条件数为 2 时下降很快，条件数为 200 时曲线明显更平缓。这里选了明确的对角矩阵与初始点，使实际误差恰好按上方的收敛因子变化；左图只是把相同几何关系旋转后展示。

这个例子告诉我们，问题的难度与矩阵的谱有关。我们接下来要改进搜索方向，让后面的移动保留前面已经完成的修正，从而更有效地利用每一步计算。

第 7 页：把“方向正交”改写成能量几何中的共轭

假设我们已经沿方向 p 完成了精确线搜索，那么沿 p 的方向导数就是 0。现在再沿另一个方向 q 移动，梯度会增加 alpha 乘 A q。原来沿 p 的方向导数，也就增加了 alpha 乘 p 转置 A q。

因此，只要让 p 转置 A q 等于 0，新的移动就会保留旧方向上的最优条件。这个关系叫作 A 共轭。它使用矩阵 A 来衡量2 个方向之间的关系，曲率也随之进入了几何定义。

右边继续使用刚才的2 维矩阵。2 个坐标轴方向在普通欧氏空间中正交，但它们的 A 内积等于 1。换成 1、2 组成的 p，以及-7、6 组成的 q，就能直接算出 A 内积为 0。

右下角还给出了一个直观读法。A 的对称正定平方根把方向变换到能量坐标，变换后2 个方向的普通内积就是原来的 A 内积。于是，共轭方向在这个坐标里成为普通的直角。共轭梯度会逐步构造这样的方向，下面我们具体算2 步。

第 8 页：共轭方向保留已经完成的修正

这一页把共轭梯度完整落在2 维例子上。初始点为 0，所以残差和第1 个搜索方向都是 b，也就是 1、2 这个向量。

第1 步的步长仍然是 1/4，得到 x_1等于 1/4、1/2。接着重算残差，得到-1/2、1/4。新残差的平方范数与旧残差平方范数之比是 1/16，这就是这里的 beta。

我们把新残差与 beta 倍的旧方向相加，得到第2 个方向。请看这个式子，它保留了上一方向的信息，同时把新出现的残差加入进来。直接代入可以验证，2 个搜索方向的 A 内积恰好等于 0。

再沿第2 个方向做线搜索，步长是 4/11。把它加到第1 步的位置，就得到 1/11、7/11，正好等于真解。右边红线画出了这2 次移动，灰色虚线是最速下降的前5 步。

这个小例子让我们看到，共轭关系怎样保留之前的优化成果。接下来把这个2 维过程推广成一个逐步扩展的搜索空间。

第 9 页：Krylov 子空间把矩阵乘法变成搜索空间

从初始残差 r_0出发，先有 r_0，再有 A 乘 r_0、A 平方乘 r_0，一直往后。前 k 个向量的所有线性组合，就构成第 k 个 Krylov 子空间。

在精确算术下，CG 的第 k 个解可以理解为：在初始点加上这个子空间的范围内，选择能量最小的点。等价地，它让 A 范数下的误差最小，而且当前残差与已经搜索的子空间正交。

右边还是2 维例子。r_0是 1、2，A r_0是 6、7。这2 个向量组成的矩阵行列式为-5，所以它们线性无关，第2 步的搜索空间已经覆盖整个2 维空间。真解可以明确写成这2 个向量的线性组合，这也解释了上一页为什么2 步就能结束。

底部再换一种表示：误差可以写成一个矩阵多项式作用于初始误差。对本例，二次多项式在2个特征值处都等于 0，因此2个特征方向上的误差同时消失。这个多项式视角，会帮助我们理解下一页的谱聚集。

第 10 页：谱聚集可以大幅加速 CG

条件数只记录最大和最小特征值之间的比例。我们再看区间内部的特征值怎样分布。左图给出 2 组谱，2 组最小值都是 1、最大值都是 1000，条件数完全相同。

红色这一组按对数间隔分布在整个区间。绿色这一组除了两端的特征值，其余118个值都集中在 100 到 101 之间。对于误差多项式来说，如果在这一小段区间里取值很小，就能同时压低很多特征方向的误差。

右图是实际迭代得到的能量误差。实验让各个特征方向的初始能量相同，再用相同的真相对残差阈值停止。聚集谱用了8步，对数分布的谱用了212步。灰色虚线是只使用条件数得到的上界。

所以，除了极端特征值，我们还要关注谱的整体结构。这里使用浮点运算，迭代次数与精确算术中的有限步结论之间的关系，会在停止标准那一页继续解释。现在先把这些几何理解落实为短递推代码。

第 11 页：CG 用短递推保存方向搜索的成果

左边是算法公式，右边是相同过程的代码。初始化时计算残差，把搜索方向设为残差，并保存残差的平方范数 rr。

每轮先计算 q 等于 A p。随后用旧残差平方范数除以 p 内积 q，得到步长 alpha。这个步长同时用于更新解和残差。接着计算新残差的平方范数，与旧值相除得到 beta，再把新残差与 beta 倍旧方向组合成下一方向。

代码用 rr 的平方根检查递推残差，因此停止判断复用了已经算好的内积。每轮核心计算包含一次稀疏矩阵向量乘法和2次内积，外加几个向量更新。我们也不需要保存所有历史方向，只维护当前这些向量。

循环结束后，代码重新计算 b 减 A x。这一步直接回到原方程检查当前解。后面还会展示 SciPy 的完整停止接口。

到这里，共轭方向让搜索过程更高效。下一步，我们进一步利用矩阵的结构，改善算法所面对的能量几何。

第 12 页：预条件通过改变坐标，改善能量几何

预条件的出发点是选择一个容易求解的对称正定矩阵 P。把它写成 C 乘 C 的转置，再做页面上的变量变换，就得到一个等价系统。这个新系统仍然对称正定，同时可能拥有更好的特征值分布。

看 2 幅等高线。原系统的2个坐标尺度差别很大，左边的等高线十分狭长。这里用对角矩阵做缩放后，右边的等高线变得更均衡，条件数从大约 134 降到了 3。

实际计算时，我们可以在原坐标中实现这个变换。预条件共轭梯度，也就是 PCG，每轮先求 z 等于 P 的逆作用在残差 r 上，再使用这个经过处理的残差更新搜索方向。收敛界对应的是变换后系统的条件数。

选择 P 时有2个同时要考虑的问题：它能多大程度改善几何，以及每次应用它需要多少计算。下面就用 2 种预条件，分别处理变量尺度与网格中的强耦合。

第 13 页：预条件同时处理尺度与耦合

我们回到最开始的网格矩阵 L，再做对称缩放，构造教学系统 A 等于 S L S。S 的对角元素沿每行从 1 逐渐增加到 20，这给系统加入了明显的变量尺度差异。

左边是 Jacobi 预条件：只取 A 的对角元。应用它的时候，每个残差分量除以对应的对角元，就得到新的分量。这个操作非常便宜，而且能有效消除尺度差异。在 32 乘 32 的网格上，条件数由大约 16895 降到 441。

右边的行块预条件进一步保留横向的强耦合。它把同一行的未知量放在一起，求解一个三对角系统。由于所有网格行使用相同的结构，长度为 m 的因子可以重复使用。这个例子里，预条件后的条件数进一步降到 5.35。

下面的流程图按顺序表达了实际计算：先除以缩放向量，再逐行求解，最后再除一次缩放向量。这里实现的是 P 的逆作用。SciPy 的 M 参数接收的正是这个操作，接下来我们看对应代码。

第 14 页：行块预条件：一个带状因子服务所有网格行

这一页把行块预条件拆成可以直接实现的步骤。因为 P 由两侧的缩放矩阵和中间的行块矩阵组成，所以它的逆作用也可以分成3步：先处理尺度，解各个行块，再恢复尺度。

左侧代码中的 band 只使用 2 行。第 1 行保存主对角元，第 2 行保存次对角元，随后对这个带状矩阵做 Cholesky 分解。我们把分解放在函数外面，因此它只需要准备一次。

进入 apply_Pinv 后，先把 r 逐项除以 s，再恢复成网格形状。这里的转置使每列对应一个网格行的右端，带状求解器就能一次处理这些右端。最后再转回原顺序，并除以 s。

右边说明 PCG 对普通 CG 递推的修改。先求 z，并以 z 初始化方向；把原来的残差平方范数替换为 r 与 z 的内积 rho。alpha 和 beta 使用 rho 更新，新的搜索方向则由新的 z 与旧方向组合。

每轮要解 m 个长度为 m 的三对角系统，总工作量与 m 平方，也就是未知量个数 n 成正比。现在我们把这 2 种预条件放到同一个系统中比较。

第 15 页：同一系统上的 4 种迭代轨迹

这组比较使用完全相同的矩阵、右端和初始点。网格大小是 32 乘 32，真解由固定随机种子生成，再用 b 等于 A 乘真解构造右端，所以实验里既能计算残差，也能计算真实的能量误差。

左图是真相对残差，右图是相对能量误差。2 幅图都使用对数坐标，方法的颜色保持一致。可以看到，普通 CG 比最速下降更快；加入对角预条件后进一步加速；保留强横向耦合的行块预条件下降最快。

下方表格给出同一停止阈值下的结果。CG 用了587步，Jacobi-PCG 用了140步，Line-PCG 用了14步。最速下降运行到10000步时，真相对残差约为10^{-5}，因此表里将达到目标的步数记为大于 10000。

这组实验把前面的 2 条机制联系起来：方向共轭改善搜索，结构化预条件改善问题本身。接下来扩大网格，看看这些收益怎样变化。

第 16 页：网格加密后的收益与成本

现在把每个方向的内部点数依次取 16、32 和 64，对应 256、1024 和4096 个未知量。其他主要构造方式和停止阈值保持一致。

左图显示 3 种方法的迭代次数都随网格加密增加。CG 从225步增加到1236步，Jacobi-PCG 从72步增加到214步，行块方法则从9步增加到23步。

右图补充了预条件后条件数的变化。行块方法的条件数从 2.14 增加到 17.94，这解释了为什么它的迭代次数也在增长。

接下来要把迭代次数换成实际工作量。每种方法都需要稀疏乘法，Jacobi 还需要逐项除法，行块则需要带状求解。总成本由准备阶段加上迭代次数乘每步成本组成。同一矩阵如果对应多个右端，准备好的预条件和分解因子还可以重复使用。

第 17 页：迭代数与每步成本，共同决定总时间

这一页直接测量4096个未知量的系统。表格把准备和求解分开，结果取本机预热后7次测量的中位数。

左图横轴采用对数尺度，点的位置表示一次求解时间。CG 大约是52毫秒，Jacobi-PCG 约11毫秒，行块 PCG 约3.7毫秒。稀疏 LU 完成分解后，一次求解大约需要0.46毫秒；它前面的准备阶段则需要约8.7毫秒。

右图用准备时间加上 R 倍单次求解时间，估算同一矩阵重复求解的总成本。随着右端数量增加，准备成本被摊薄，复用分解因子的收益更明显。

因此，求解器选择需要同时看矩阵规模、每步操作和右端数量。迭代次数提供了重要线索，而实际时间把预条件应用与因子准备这些工作也一起算了进去。最后，我们还需要可靠地判断一次求解什么时候完成。

第 18 页：用真残差停止，再核对误差尺度

先看左边。SciPy 接口中的相对阈值与绝对阈值共同确定收敛标准。返回后重算真残差，按它的范数不大于相对阈值乘右端范数与绝对阈值中的较大者进行验收。相对阈值控制相对于右端尺度的精度，绝对阈值提供固定的残差尺度。

右边把残差与解误差联系起来。相对解误差的一个通用上界，是条件数乘以相对残差。对于这里条件数约 16900 的系统，相对残差达到10^{-8}，对应的这个上界约为1.69×10^{-4}。

下面代码把行块的逆作用包装成 LinearOperator，传给 cg 的 M 参数。返回后，我们同时检查 info 和重新计算的残差。本例 info 等于 0，真相对残差约为9.05×10^{-9}。

最后回到第 10 页的120维例子。精确算术中，CG 至多经过 n 步结束；浮点运算中的舍入会逐渐损失共轭性，因此那个例子用了212步。递推残差方便每一步更新，最后重算真残差则直接检查原方程。

第 19 页：直接实现五点乘法，也能接入同一个 CG 接口

到这里，我们一直把 A 看作显式存储的稀疏矩阵。CG 实际使用 A 的关键操作，是矩阵乘向量。对于最开始的五点结构，可以直接写出这个操作。

看左边的函数。先将向量按 s 缩放，并恢复为2维网格 U。V 先接收中心点的贡献，然后通过 4 组切片累加左右、上下邻点的贡献，最后再缩放并展平成向量。

这些切片只处理内部邻点。当前采用零边界条件，边界外的场值为 0，所以这一实现与原来的五点矩阵对应。把函数包装成 LinearOperator 后，下面的 cg 调用仍然使用相同的预条件接口。

右侧给出了实际核对：同一测试向量下，算子乘法与显式矩阵乘法的相对差约为1.67×10^{-16}；配合行块预条件，真相对残差达到9.05×10^{-9}。

这一步把矩阵的局部结构直接转成了计算过程。算法接口、预条件与残差检查都得以保留，存储主要用于向量和预条件数据。

第 20 页：把结构、收敛与成本连成一条求解路径

最后把整条路线合起来。面对对称正定的稀疏系统，可以从 CG 或 PCG 出发；尺度差异明显时，先考虑对角均衡；存在强方向耦合或变量组结构时，再考虑行块或其他分块方式。多个右端则需要把因子复用纳入总成本。

今天最重要的3个联系是：条件数把能量几何连接到收敛尺度，共轭方向保留已经完成的修正，预条件把矩阵结构转化为更有效的迭代。

从网格上的局部方程，到能够运行并检查结果的求解程序，我们始终围绕这 3 条线索选择表示、算法与实现。谢谢大家。